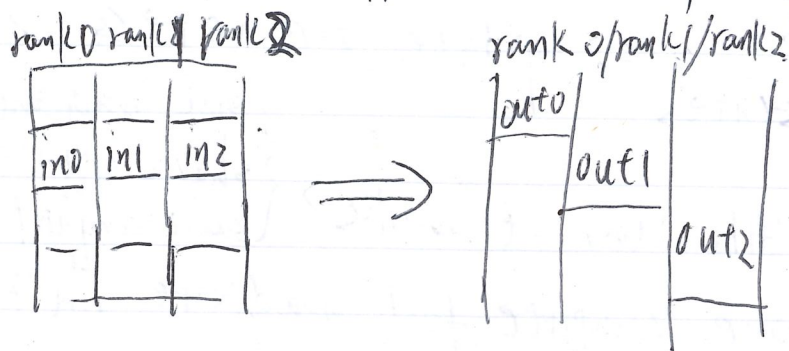


Gather: collect data from all GPUs into one GPU

All-Gather: Each GPU collects data from ^{different pieces} all GPUs, ending up with the full combined dataset.

Reduce-Scatter: combine (reduce) values from all ^{GPUs} ranks, then give each rank a different slice of the reduced result.



All-to-All: Each GPU exchange unique data with all other GPU.

Barrier: Synchronize all ranks/GPUs — no one proceeds until everyone reaches the same point

Zero stage ① is free (in bandwidth limited regime) memory wins
Zero stage ② the simple extension to gradient sharding

Idea: keep the gradients (ppl's slices) sharded across machines
use the same (rough) tricks as stage 1.

Zero stage ③ => AllReduce

idea: { shard everything: params/gradients/local states
same 'incremental communication/computation' ideas
send and request params on demand while stepping through compute graph

How stage-3 based FSDP works

1: incremental computation/communication

=> params/ gradients are requested/sent and are immediately freed

2: overlapping communication and computation $(w_1 w_2 w_3) X = Y$

=> The all-gathers happen all at once while forward happens, masking the comm cost.

Summary: Distributed Data Parallel cost $2^* \# \text{param}$ communication ^{communicate}

Zero stage #1: $2^* \# \text{param}$: shard optimizer states = low overhead

stage #2: $2^* \# \text{param}$: shard opti states + gradients

stage #3: $2^* \# \text{param}$: shard opti states + gradients + params

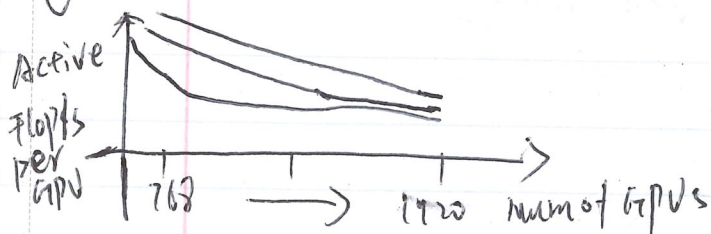
FSDP $\rightarrow$ each GPU stores entire $\left\{ \begin{array}{l} \text{high memory} \\ \text{efficiency} \\ \text{high comm overhead} \end{array} \right.$

Issue with data parallelism - compute scaling:

with data parallel, # of machines < batch size (comm overhead is high)

Stages #1 and #2 can't scale memory

(stage #3 can be slow and doesn't reduce activation memory)



$\downarrow$
Beyond Data parallelise: model parallelism.